

Kapitel 1

Schaltungsentwurf

Schon in der Vorbereitungsphase dieser Arbeit wurde die Schaltung aus Experiment 5 auf dem ASLK-PRO-Board aufgebaut. Dabei fiel auf, dass am SF-Pin des Multiplizierers statt der im Datenblatt angegebenen 10 V nur etwa 8,78 V anliegen. Der Grund dafür ist die Höhe der Versorgungsspannung, die auf dem ASLK-PRO-Board nur bei ± 10 V liegt, anstatt der im Datenblatt vorgeschlagenden ± 15 V. Dieser Unterschied sollte die Funktion des Multiplizierers zwar nicht beeinträchtigen, für eine bessere Verständlichkeit der Schaltung wären die ± 15 V aber hilfreich. Der verwendete opv TL082B ist laut Datenblatt bis zu ± 20 V verwendbar, sodass das erste pcb mit ± 15 V Versorgungsspannung geplant wird.

1.1 Design des Schaltplans

Im Folgenden wird die Entwicklung des Schaltplans genauer beschrieben. Die abgebildeten Teilschaltungen zeigen dabei immer die zweite, finale Iteration des pcb. Leider kann der Schaltplan aufgrund der Größe und der damit zusammenhängenden Lesbarkeit nicht in einer einzigen Abbildung dargestellt werden. Stattdessen werden die einzelnen Module und Teilschaltungen in separaten Abbildungen gezeigt und erläutert. Für die Gesamtübersicht wird auf den im Repositorium hinterlegten Schaltplan verwiesen. **Ein Link zum GitHub Repositorium befindet sich im Anhang.**

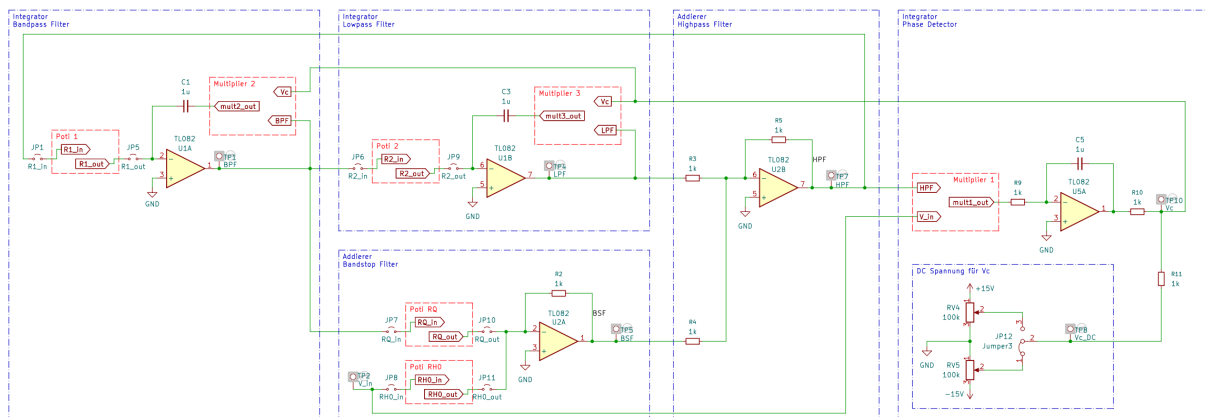


Abbildung 1.1: Darstellung des VCF ohne Peripherie

soll die abbildung raus?

Die meisten der im System verwendeten Bauteile werden aus dem Aufbau in ANS bzw. dem ALSK-PRO Manual [1] übernommen. So wird als opv wieder der TL082 eingeplant, wobei darauf geachtet wird, den für Filteranwendungen etwas besseren TL082B zu verwenden. Dieser basiert auf der gleichen Architektur, bietet aber leicht verbesserte Werte im Bereich der Input Offset Voltage und des Input Offset Drifts. Dadurch werden besonders im Bereich der Integration Fehler minimiert, sodass diese Version etwas präziser arbeiten sollte. Das gbw und die Slew-Rate werden durch die Wahl der Version nicht beeinflusst.

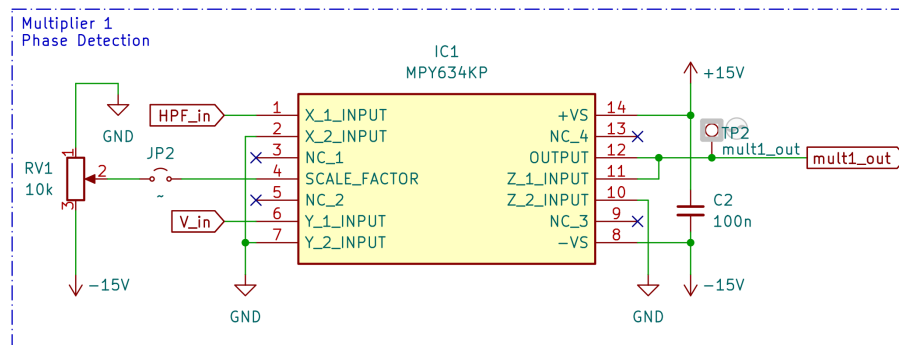


Abbildung 1.2: Verschaltung des Analog-Multiplizierers MPY634

In Abbildung 1.2 ist die Verschaltung des ersten von drei analogen Multiplizierern zu sehen. Die weiteren beiden Bausteine werden abseits der Ein- und Ausgänge identisch verschaltet, sodass die resultierende Übertragungsfunktion der Multiplizierer der Gleichung ?? aus Kapitel ?? entspricht. Dabei kann das Potentiometer am rechten Rand der Abbildung zur Einstellung der Referenzspannung V_r innerhalb des Multiplizierers verwendet werden. Durch das Entfernen des Jumpers kann diese Funktion aber auch deaktiviert werden, wodurch die internen -10 V anliegen sollten. Der 100 nF -Kondensator zwischen den Versorgungsspannungen dient der Glättung von Störspannungen.

Für die Digitalpotentiometer fällt die Wahl auf das von Herrn Ziemann vorgeschlagene MCP4261. Bei der weiteren Auswahl wird Wert auf eine möglichst hohe Schrittzahl und einen angemessenen Maximalwert gelegt. Da zur Einstellung des Güte- und Verstärkungsfaktors sowie der Mittenfrequenz des Filters insgesamt vier Potentiometer gebraucht werden, wird ein Modul gewählt, bei dem sich zwei regelbare Widerstände in einem Gehäuse befinden. Der MCP4261 ist zudem als Potentiometer und Rheostat erhältlich, wobei das Rheostat die gewünschten strombegrenzenden Eigenschaften besitzt, während das Potentiometer eine Spannungsteilung ermöglicht. Trotzdem wird in diesem Fall die Potentiometerversion verwendet, da diese einfacher in PDIP-Gehäusen erhältlich ist und sich durch das Offenlassen eines Pins (nicht der Abgriff) als verstellbarer Widerstand einsetzen lässt.

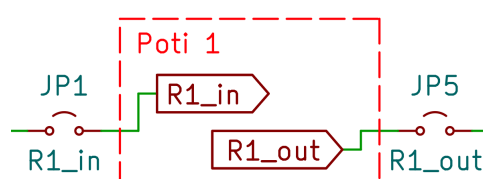


Abbildung 1.3: Darstellung der Ein- und Ausgänge der Digitalpotentiometer

Die Ein- und Ausgänge der Potentiometers werden, wie in Abbildung 1.3 zu sehen, jeweils mit Jumpers versehen, damit der eingestellte Widerstandswert schnell und ohne Beeinflussung durch die restliche Schaltung gemessen werden kann.

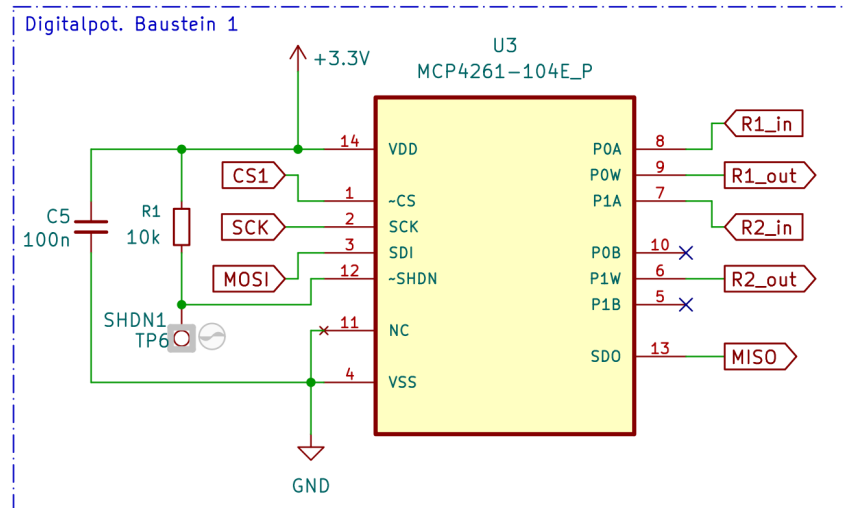


Abbildung 1.4: Verschaltung der Digitalpotentiometer MCP4261

Abbildung 1.4 zeigt die Anschlüsse des ersten Digitalpotentiometers. Das zweite Modul ist dazu identisch und die Verschaltung erfolgt wie im Datenblatt angegeben. Dazu gehören die Anschlüsse zum spi-Bus sowie des Glättungskondensators zwischen V_{SS} und V_{DD} und einem 10 k Ω Pull-Up Widerstand um den SHDN-Pin auf ein definiertes High zu ziehen. Laut Datenblatt soll der non-connected Pin auf GND oder Versorgungsspannung gesetzt werden. Das Schaltsymbol für den MCP4261 wird dabei nicht selbst erstellt, sondern von Mouser heruntergeladen und in eine eigene Bibliothek eingefügt. Problematisch ist dabei, dass die Zuteilung der Pins für die internen Potentiometer nicht ideal im Symbol wiedergegeben wird. Pin P1A und Pin P0B sind im Schaltsymbol vertauscht, was berücksichtigt bzw. überarbeitet werden musste.

Zum Zeitpunkt der Schaltplanerstellung ist die Funktion der Hilfsspannung V_H noch nicht genau bekannt. Damit später trotzdem der optimale Wert einstellbar ist, soll die in Abbildung 1.5 zu sehende Schaltung einen Spannungswert zwischen ± 15 V ausgeben können.

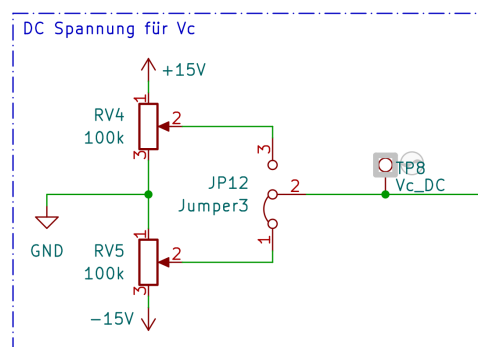


Abbildung 1.5: Implementierung der einstellbaren Hilfsspannung V_H

Um die Signale der verschiedenen Filtertypen besser mit geeigneten Messinstrumenten wie einem Oszilloskop oder Spektrumanalysator aufnehmen zu können, werden bnc-Anschlüsse auf der Platine angebracht, da dieser Standard bei Messgeräten weit verbreitet ist.

Auf dem pcb übernimmt die mcu die Ansteuerung der Digitalpotentiometer. Später soll diese zudem ein ui zur Bedienung der Filterparameter bereitstellen. Hierfür wird ein Raspberry Pico 2 W verwendet, dessen Mikrokontroller RP2350 in Europa entworfen wurde. Er ist eine Weiterentwicklung des RP2040 und sollte für diese Aufgabe leistungsstark genug sein. Zusätzlich befindet sich auf dem Pico-2-Modul der wlan-Baustein CYW43439 von Infineon, der die drahtlose Kommunikation (WLAN und Bluetooth) zwischen Eingabegerät und Filter ermöglicht.

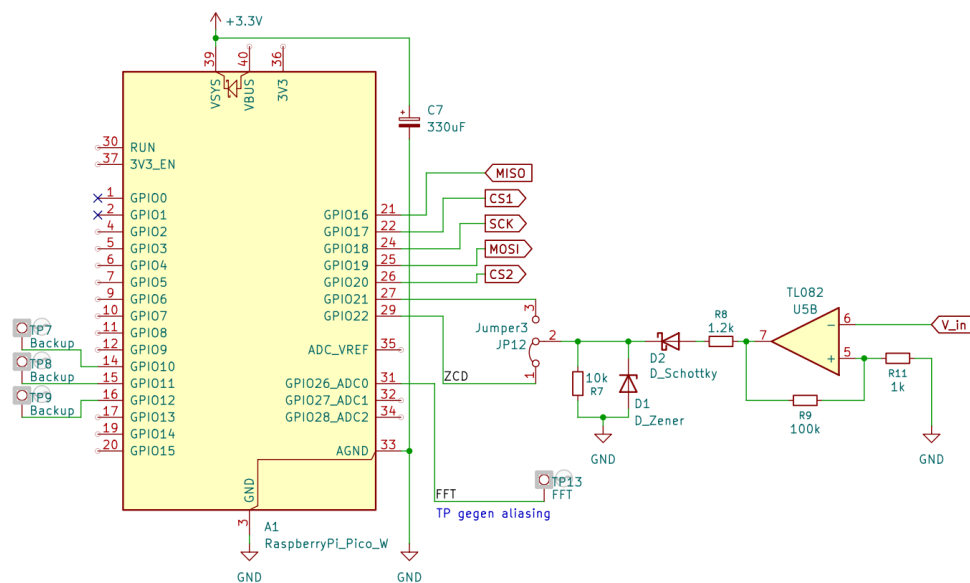


Abbildung 1.6: Verschaltung des Raspberry Pico 2 W

Ebenfalls in Abbildung 1.6 zu sehen ist ein weiterer opv, der zur Frequenzbestimmung des Eingangssignals genutzt werden soll. Der opv wird als invertierender Schmitt-Trigger beschaltet, um das sinusförmige Eingangssignal in ein Rechtecksignal zu überführen. Da die Ausgangsamplitude des Schmitt-Triggers der Versorgungsspannung von $\pm 15\text{ V}$ entspricht, ist eine Pegelanpassung für die mcu zwingend erforderlich.

Zur Strombegrenzung wird nach dem opv-Ausgang ein Widerstand in Reihe geschaltet. Die anschließende Diodenschaltung begrenzt das Signal auf einen Logikpegel-kompatiblen Bereich von 0 V bis 3,3 V. Die Schottky-Diode in Reihe kappt die negativen Halbwellen, während die parallel geschaltete 3,3 V-Zener-Diode alle Spannungen über 3,3 V gegen Ground ableitet. Der externe Pull-Down Widerstand sorgt für ein definiertes Groundsignal während der negativen Halbwelle. Im Vergleich zum internen Pull-Down-Widerstand des RP2350, welcher mit ca. 50 k Ω relativ hochohmig ist, ermöglicht der externe Widerstand ein schnelleres Entladen der parasitären Kapazitäten am gpio-Pin und sorgt so für sauberere Schaltflanken.

Die Frequenzbestimmung in der mcu soll mittels der Nulldurchgangsmethode erfolgen. Um die Flexibilität des pcbs zu erhöhen, sind zusätzliche gpio-Pins über Testpunkte oder

Jumper zugänglich. Dies dient sowohl der Fehlersuche als auch der optionalen Erweiterung des Systems, beispielsweise durch eine hochauflösendere Frequenzanalyse via adc.

Sowohl die Digitalpotentiometer als auch die mcu benötigen eine Versorgungsspannung im Bereich von etwa 1,8 V bis 5,5 V. Da auch Busse und Pins der mcu mit 3,3 V betrieben werden, wird eine 3,3 V-Versorgungsspannung integriert. Um dieses Potential zu erreichen ohne hohe Verluste zu generieren, soll ein Buck-Converter die 15 V auf ein Niveau von 3,3 V absenken. Die zugehörigen externen Bauteile des Buck-Converters werden wie im Datenblatt angegeben anhand des Maximalstroms, der maximalen Eingangsspannung und der Ausgangsspannung dimensioniert. Zu sehen ist diese Schaltung in Abbildung 1.7.

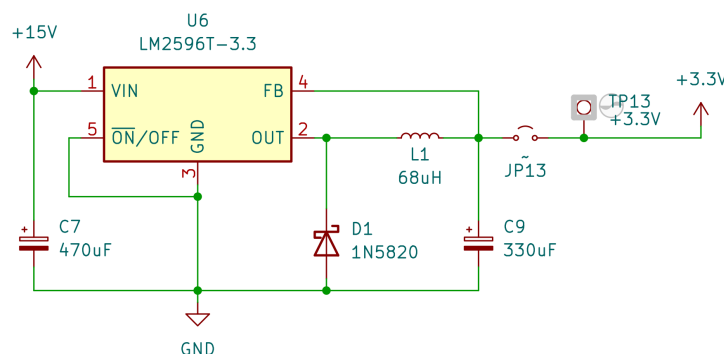


Abbildung 1.7: Aufbau des Buck-Converters

bild aktualisieren und beschreibung überarbeiten

Zuletzt wird überprüft, ob alle relevanten Signalfade mit Testpunkten versehen sind, um während der Messungen möglichst viele interne Signale des Self-Tuned Filters aufnehmen zu können. Das soll später dabei helfen potentielle Fehler schneller zu identifizieren. Zusätzlich werden Mounting-Holes gesetzt, damit das pcb sicher fixiert werden kann und der Zugriff auf die Edge-Mount-bnc-Steckverbinder problemlos funktioniert. Insgesamt wird der Schaltplan so gestaltet, dass er möglichst übersichtlich und gut nachvollziehbar bleibt.

1.2 Design der Leiterplatte (PCB)

Beim Platinendesign muss zuerst entschieden werden, wie viele Lagen das pcb haben soll. Grundsätzlich kann jeder beliebige Schaltplan auf einem zweilagigen pcb umgesetzt werden. Dies geht allerdings auf Kosten der Platinengröße und kann Störeinflüsse begünstigen. Um beide Aspekte zu minimieren, wird ein vierlagiges Layout gewählt, da dies das Routing erheblich vereinfacht und die Mehrkosten überschaubar sind.

Ein wesentlicher Vorteil von mehrlagigen pcb besteht darin, dass Bauteile deutlich dichter aneinander platziert werden können, ohne dass sich dazwischenliegende Leiterbahnen gegenseitig behindern. Dieser Vorteil wirkt sich noch stärker auf pcb mit smd-Komponenten aus, da der Footprint nur auf der ersten Kupferlage zu erkennen ist und die anderen Lagen nicht aktiv beeinflusst. Trotzdem wurde in dieser Arbeit absichtlich auf smd-Komponenten verzichtet, da tht-Elemente im Umgang einfacher und bei Anpassungen leichter auszutauschen sind.

Die allgemeinen Aufgaben der vier Lagen werden wie folgt zugeordnet:

1. Layer 1: Signallayer

Die oberste Lage des pcb dient dem Signalrouting. Hierbei sollen, soweit möglich, alle an der Filterung beteiligten Signalfade über diese Ebene gehen. Auch wenn der in dieser Arbeit betrachtete Frequenzbereich noch recht unanfällig für Störungen ist, wird darauf geachtet, diese Störeinflüsse so gering wie möglich zu halten.

2. Layer 2: Groundlayer

Direkt darunter befindet sich eine durchgehende Massefläche. Diese Schicht bleibt abgesehen von Vias ununterbrochen, sodass das Ground-Potential über die tht-Pins beziehungsweise Vias an jeder Stelle des pcb erreichbar ist. Zudem schirmt diese Fläche potenziell störende Bus-Signale ab.

3. Layer 3 Powerlayer

Diese Schicht dient ausschließlich der Spannungsversorgung der Bauteile auf dem pcb. Die -15 V sowie die $3,3\text{ V}$ werden dabei ganz normal geroutet. Die 15 V verlaufen hingegen großflächig über die Ebene, da dieses Potential sowohl die opv betreibt, als auch die $3,3\text{ V}$ von diesem Signal aus abgehen. Dies ist die einzige Ebene in der die Zone nicht auf Ground gelegt wird. Durch diesen Aufbau von Ground- und Power-Layer bildet sich zwischen diesen beiden Flächen ein kleiner Kondensator, der die Versorgungsspannung glättet.(warscheinlich zu klein zum stabilisieren)

4. Layer 4 Signallayer

Die unterste Schicht dient als Ausweichmöglichkeit für sich kreuzende Signale. Ansonsten soll diese hauptsächlich für das Routing von Bus-Signalen verwendet werden.

Bei der Platzierung der Komponenten wird darauf geachtet, das Kommunikationsmodul der mcu ganz an den Rand des pcb zu setzen, um die Abschirmung der Antenne durch die Kupferflächen der Platine zu vermeiden. Bei der Platzierung des Buck-Converters und dazugehörigen weiteren Komponenten wird darauf geachtet, dass diese wie im Datenblatt beschrieben auf dem pcb platziert werden. Aufgrund von Platzmangel hat sich das Layout dennoch etwas verändert.

Da in der linken oberen Ecke des pcb noch etwas Platz ist, wird für einen einfachen Zugang auf die Dokumentation des Projekts ein QR-Code zum Git-Repository der Arbeit hinzugefügt.

1.3 Design des Skripts für die Steuerung

Bei der Entwicklung der Steuersoftware für die mcu muss die Vorgehensweise aufgrund geringer Vorerfahrung mit dem RP2350 und der Programmierung in MicroPython gut geplant werden.

Nach der Installation der aktuellen Micropython-Firmware auf der verwendeten mcu soll zuerst das spi-Interface initialisiert werden, um die Digitalpotentiometer ansteuern

zu können. Dafür wird der spi-Bus im Code so konfiguriert, dass MISO, MOSI und SCK den geplanten gpio-Pins zugeteilt werden. Anschließend werden die CS-Pins als normale gpio-Pins definiert. So können nun Funktionen geschrieben werden, welche die MCP4261-Befehlswörter senden um die Widerstandswerte der Potentiometer zu setzen.

Das Digitalpotentiometer verwendet bei der Kommunikation über SPI standardmäßig 16-Bit-Datenwörter. Dabei bilden die ersten 8 Bits das „Command Byte“, das die Registeradresse und den Command (den jeweiligen Wiper und seine Funktion) beinhaltet. Das darauffolgende „Data Byte“ liefert den neuen 8-Bit-Wiperwert.

Neben der Programmierung der Widerstandswerte für das ram-Register des Chips werden zudem nichtflüchtige Start- und Resetwerte im eeprom des Digitalpotentiometers hinterlegt. Diese dienen dazu, dass die Potentiometer nach einem Power-Up oder Reset des Systems auf einen definierten Anfangswert gesetzt werden, anstatt sich auf die Mittelstellung einzustellen. Falls es Komplikationen bei der Messung mit der mcu auf der Platine geben sollte, können so die gewollten Standartwerte sichergestellt werden. Da die Anzahl der Schreibzyklen des eeprom begrenzt ist, wird dieses Register nicht für das laufende Einstellen der Werte, sondern nur für die initiale Konfiguration verwendet.

Zur Fehlersuche dient zudem eine Funktion, welche die aktuellen Registerwerte der Potentiometer ausliest, um die Kommunikation auf Fehlerfreiheit zu prüfen.

Anschließend wird der Fokus auf die Umwandlung vom Filterparameter zum beschriebenen Bitwert gelegt, der den Widerstandswert des Potentiometers repräsentiert. Dafür müssen die bekannten Formeln aus dem ASLK-PRO Manual [1] in Funktionen eingebaut werden. Diese nehmen die gewünschten Filterparameter als Eingabe entgegen und berechnen den entsprechenden Wiperwert für das jeweilige Potentiometer. Innerhalb der Funktionen wird sichergestellt, dass die Werte immer im gültigen Bereich von 0 bis 255 liegen. Zudem wird der ausgegebene Bitwert gerundet. Zur besseren Kontrolle errechnet eine weitere Funktion die tatsächlich anliegenden Widerstandswerte aus den gerundeten Bitwerten.

Darauf folgend wird die wlan-Schnittstelle der mcu eingerichtet. Der Pico soll hierbei automatisch eine Verbindung mit dem angegebenen wlan aufbauen, um die Filterparameter plattformunabhängig über einen http-Server im Browser eines PCs (im gleichen Netzwerk) anpassen zu können. Da für die html-Programmierung keine Vorkenntnisse vorhanden sind, wird dieser Teil mit Hilfe eines llm umgesetzt. (Kapitel: ??) Der vom Pico gehostete Server generiert hierbei eine html-Seite, die neben den Eingabefeldern für Frequenz, Güte und Verstärkung auch eine tabellarische Übersicht der aktuellen Zustände enthält. Diese Tabelle wird im Laufe der Arbeit noch erweitert, um eine bessere Übersicht über die Stellung der Wiper und deren Genauigkeit zu erhalten.

Damit das System nach Anlegen der Versorgungsspannung selbstständig startet, ohne den Code in Thonny ausführen zu müssen, wird das Skript als main.py im Flash-Speicher der mcu hinterlegt. Für die eigenständige Nutzung des Systems ist es zudem wichtig, dass der Server unter einer IP-Adresse erreichbar ist, die sich bei Neustart nicht ändert. Leider sind Protokolle wie mDNS nicht auf der mcu unter MicroPython verfügbar, weswegen der Pico im Access-Point-Modus betrieben wird. Hierbei erzeugt der Pico ein eigenes lokales Netzwerk mit der SSID „Pico-Filter“. Durch die Konfiguration einer statischen IP-Adresse (192.168.4.1) ist gewährleistet, dass das Steuergerät den HTTP-Server nach der Verbindung mit dem Netzwerk jederzeit unter derselben Adresse erreicht.

1.3.1 Frequenzdetektion über Nulldurchgangszähler

Nach der Implementierung der Filtersteuerung soll der Code um eine Frequenzmessung erweitert werden. Dafür detektiert der als Schmitt-Trigger verschaltete opv hardwareseitig den Nulldurchgang des Eingangssignals. Das daraus resultierende Rechtecksignal wird auf 0 bis 3,3 V gekappt, sodass im Code der mcu ein Rising Edge Counter programmiert werden kann. Dieser zählt die steigenden Flanken innerhalb eines angegebenen Zeitbereichs und gibt das Ergebnis in Hertz aus.

In der ersten Testphase wurde der Zähler über eine Interrupt Service Routine realisiert, bei der jede steigende Flanke einen Interrupt auslöst, der wiederum einen Zähler hochtreibt. Diese Umsetzung eines Zählers belastet die CPU jedoch stark, was zu spürbaren Verzögerungen innerhalb der Weboberfläche führt.

Um dieses Problem zu lösen, wird die Messung auf eine der pio-Einheiten des RP2350 ausgelagert. Diese Hardware-Subsysteme arbeiten unabhängig von der CPU und verfügen über eine ebenso hohe Taktfrequenz. Der PIO-Zustandsautomat zählt so eigenständig die Flanken und speichert sie in einem Register ab. Anschließend kann die CPU die Werte aus dem Register auslesen, ohne dass es in der Weboberfläche zu Verzögerungen kommt. Auch diese Implimentierung wurde über ein llm umgesetzt. (Kapitel: ??)

Bei der Überprüfung der Frequenzmessung des finalen Zählers fallen im Vergleich zur Eingangsfrequenz starke Abweichungen auf. Deswegen wurde, wie in Tabelle 1.1 zu sehen, eine kleine Messreihe aufgenommen, um den Fehler einzugrenzen. Die rsultierenden Messwerte werden auf 1 Hz gerundet, um geringe Schwankungen zu unterdrücken.

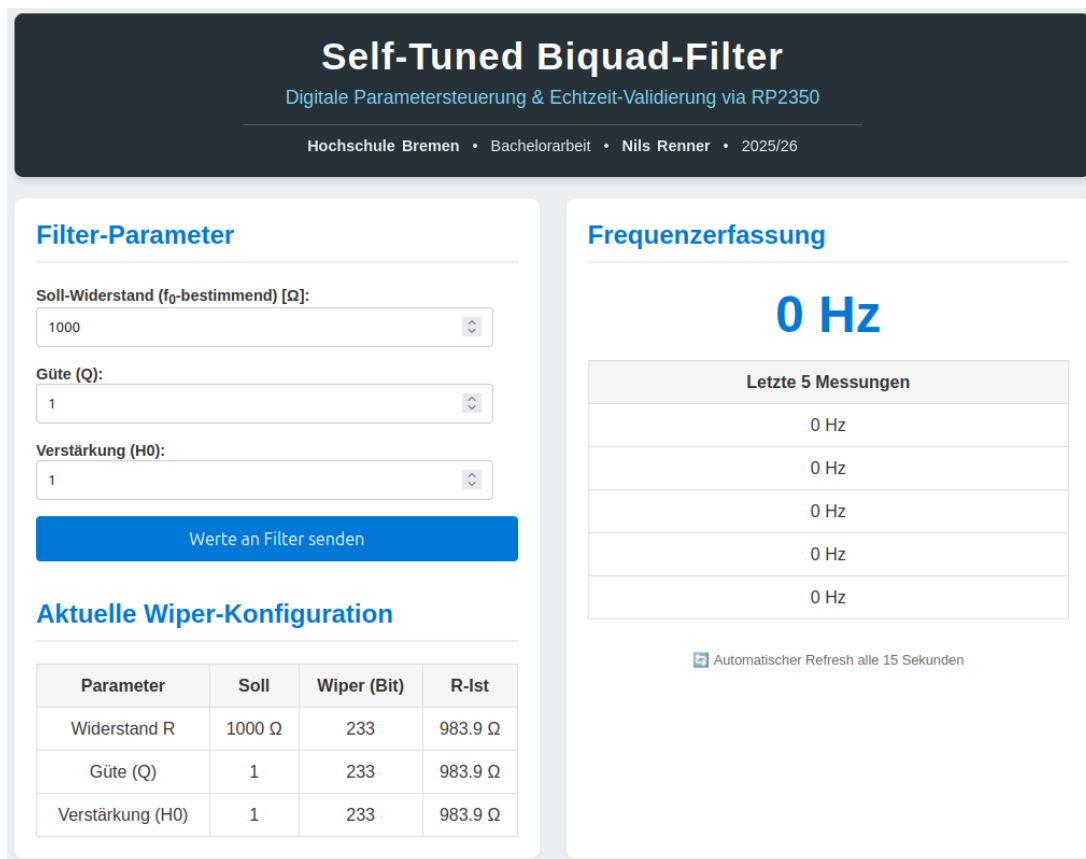
Eingangsfrequenz / Hz	alte Messung / Hz	Abweichung / %	neue Messung / Hz
100	101	1.00	100
500	503	0.6	500
1000	1007	0.7	1000
5000	5035	0.7	5000
10000	10071	0.71	10001
15000	15106	0.71	15001
20000	20142	0.71	20001

Tabelle 1.1: Abweichung der Frequenzdetektion vor und nach der Korrektur

Dabei fällt auf, dass die Abweichung linear zur Frequenz steigt und der prozentuale Fehler konstant bleibt. Dies lässt darauf schließen, dass der interne Takt der mcu von der Nominalfrequenz abweicht. Zur Kompensation dieses Fehlers wird ein Vorfaktor in den Code eingebaut, der die berechnete Frequenz mit dem Faktor 1,007 multipliziert. Diese Multiplikation wirkt zunächst unintuitiv, da die Messwerte bereits leicht erhöht sind, sie begründet sich jedoch in der speziellen Funktionsweise der pio. Da der genutzte Zustandsautomat das Register nicht hochzählen kann, sondern bei jeder detektierten Flanke herunterzählt, muss die Differenzbildung zum Startwert im Code mathematisch korrigiert werden. Die Ergebnisse dieser Korrektur sind ebenfalls in der Tabelle dargestellt.

1.3.2 Vorstellung des User Interface

Nach Fertigstellung der finalen Skriptstruktur erfolgt die Überarbeitung der ui. Dabei soll die Eingabe so angepasst werden, dass der Wert der frequenzbestimmenden Widerstände innerhalb Filterparameter angegeben wird, anstatt die Frequenz selber einzustellen. In der ursprünglichen Umsetzung konnte die Zielfrequenz über die Formel $f_0 = \frac{1}{2\pi \cdot RC}$ angegeben werden. Für den finalen Schaltungsaufbau des Self-Tuned Filters ist diese Angabe jedoch wenig zielführend, da die Resonanzfrequenz zusätzlich von Parametern wie der Referenz- und der Steuerspannung abhängt.



Self-Tuned Biquad-Filter
 Digitale Parametersteuerung & Echtzeit-Validierung via RP2350
 Hochschule Bremen • Bachelorarbeit • Nils Renner • 2025/26

Filter-Parameter

Soll-Widerstand (f_0 -bestimmend) [Ω]:
 1000

Güte (Q):
 1

Verstärkung (H_0):
 1

Werte an Filter senden

Aktuelle Wiper-Konfiguration

Parameter	Soll	Wiper (Bit)	R-Ist
Widerstand R	1000 Ω	233	983.9 Ω
Güte (Q)	1	233	983.9 Ω
Verstärkung (H_0)	1	233	983.9 Ω

Frequenzerfassung

0 Hz

Letzte 5 Messungen

0 Hz
0 Hz
0 Hz
0 Hz
0 Hz

Automatischer Refresh alle 15 Sekunden

Abbildung 1.8: User-Interface des Filters

In Abbildung 1.8 ist die finale Version der Weboberfläche dargestellt. Die linke Spalte umfasst die Eingabefelder für den frequenzgebenden Sollwiderstand, den Gütefaktor sowie den Verstärkungsfaktor. Unterhalb der Filterparameter befindet sich eine Tabelle zur aktuellen Wiper-Konfiguration, die die auftretende Abweichung aufgrund von Rundungsfehlern und der Schrittweite der Digitalpotentiometer dokumentiert.

Auf der rechten Seite der Weboberfläche befindet sich die Eingangsfrequenzdetektion. Dabei werden neben dem aktuellen Messwert die letzten 5 Messungen angezeigt, um kurzzeitige Schwankungen besser beurteilen zu können. Um eventuelle Änderungen an der Eingangsfrequenz anzuzeigen aktualisiert sich die Website alle 15 s.

Bei der Implementierung der Frequenzdetektion fiel auf, dass die Resonanzfrequenz des Filters auch über das Potential der Steuerspannung V_c bestimmt werden kann. Über diesen

Ansatz könnte die gemessene Eingangsfrequenz direkt mit der physikalisch eingestellten Resonanzfrequenz verglichen werden. Problematisch ist hierbei jedoch, dass V_c den maximal zulässigen Pegel der mcu-Eingangspins von 3,3 V ab einer Frequenz von unter 483 Hz überschreitet, weswegen eine zusätzliche Hardware-Vorverarbeitung erforderlich wäre. Die Umsetzung dieser Resonanzfrequenzdetektion wäre eine mögliche Erweiterung für eine zukünftige Systemiteration.